Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 4

Due: March 22, 2026

Please note the submission rules at the end of this exercise sheet. Non-adherence to these rules may lead to your submission not being marked.

Exercise 4.1 (3.5+1.5+0.5+0.5 marks)

The task in this exercise is to write a Java program. We expect you to implement your code on your own without using existing code or automatically generated code. If you encounter technical problems or have other difficulties with the task, please contact your tutor as early as possible.
The objective of Exercise 2.4 was to implement the state space of the pancake problem. On ADAM, you'll find Java code associated with this exercise sheet (`iterative-deepening.zip`) that implements an interface for search algorithms in the base class `SearchAlgorithmBase`. It builds on the model solution of Exercise 2.4, but you can replace the state space with your own solution if you prefer. For this exercise, you only have to create a new file `IterativeDeepeningSearch.java`.

(a) Implement iterative deepening search in a new file `IterativeDeepeningSearch.java`. Your class must inherit from `SearchAlgorithmBase` and implement the method `run()`. Print the number of generated nodes (i.e., the number of recursive calls to depth-limited search) for each depth level as well as the total number of generated nodes.

(b) Test your implementation on the problem instances you can find on ADAM with this exercise sheet. Set a CPU time limit of 10 minutes and a memory limit of 2 GB for each run.

To set a CPU time limit on Linux, use the command `ulimit -St 600`. This will set a soft limit of 600 seconds, after which your Java process will receive a signal to terminate. You can recognize that the task was terminated by the signal by inspecting its exit code. The signal for a time limit is `SIGXCPU`, which will be reported as an exit code of 152. Note that we are limiting CPU time, not wall clock time. Your actual run could be much faster (if Java internally uses multiple cores) or even slower (if other processes are running on your computer). We prefer measuring CPU time over wall clock time, as it gives more reproducible results.

To set the memory limit, run your instances with the parameter `-Xmx2048M`. For example, the first instance can be run like this:

```
java -Xmx2048M IterativeDeepeningSearch pancakes instances/pancakes_prob_01
```

Report runtime, number of generated nodes in the last iteration, total number of generated nodes, and solution length for all instances that can be solved within the given time and memory limits. For all other instances, report whether the time or the memory limit was violated in the column "result", and report the values of the last statistics output (generated nodes in last iteration and in total and lower bound on solution length) with an added ">".

Example:

			generated nodes		
instance	result	runtime	last iteration	total	solution length
01	solved	0.02	5 429	7 294	5
02	timeout		> 19 608	> 22 875	> 5
...					

(c) Problems `01` and `03` have the same solution lengths and therefore find solutions in the same iteration of iterative deepening search. Yet, they generate significantly different numbers of nodes before finding that solution. How is this possible given that both solve instances of the pancake problem?

Hint: What is the main difference between the two problem instances and how does this influence the induced state spaces?

(d) Does your experiment from part (b) confirm the theoretical claim that the total number of generated nodes is dominated by the last iteration? Justify your answer.

Exercise 4.2 (2 marks)

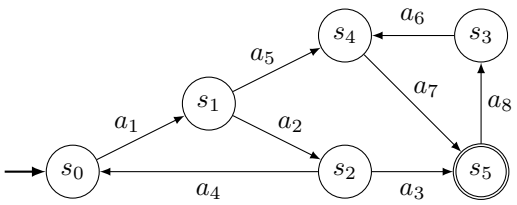
Consider an instance of the pancakes problem with n pancakes where a state $s = \langle p_1, \ldots, p_n \rangle$ represents the sizes of the pancakes from top to bottom. Now consider the following heuristic:

$$h(s) = \sum_{i=1}^{n-1} (|p_i - p_{i+1}| - 1)$$

Intuitively, for each pair of adjacent pancakes p_i, p_{i+1}, we count how many pancakes we still need to put in between them to have all pancakes in order.
Which of the four properties *safe*, *goal-aware*, *admissible* and *consistent* does h satisfy? Justify your answer for each property.

Exercise 4.3 (2 marks)

For the state space depicted below with uniform action costs of 1, define a heuristic that is consistent, not safe and has a perfect estimate for s_0 (i.e., $h(s_0) = h^*(s_0)$). Justify your answer by showing why the heuristic is consistent and not safe.
Hint: If a consistent heuristic assigns ∞ to a state, all its successors must be assigned ∞ as well.



Submission rules:

- Submit one solution in groups of up to two students, stating the names of all group members in the submission. All group members are equally responsible for all submitted exercises.
- Create a single A4 PDF file for all non-programming exercises. Solutions in LaTeX are preferred, but clearly legible scans in A4 with reasonable contrast and file size are accepted as well.
- In case there are programming exercises, upload a ZIP file (ending .zip, .tar.gz or .tgz; *not* .rar or anything else) including your PDF and those code textfiles required by the exercise. Put your names in a comment on top of each code file. Make sure your code compiles and test it. Code that does not compile or which we cannot successfully execute will not be graded.
- Include a statement on scientific integrity at the end of your submission (see below).

Statement on scientific integrity:

- All submitted solutions must be the product of your own work.
- If your written solutions were produced using any help (e.g., from peers or a large language model), list in what way these people or tools contributed to the solutions.
- We do not mark submissions that lack a statement on scientific integrity.
- Examples:
  - "We solved all exercises on our own."
  - "We used ChatGPT to explain exercise 7 which we did not understand."
  - "We had no time to solve this exercise sheet and copied all solutions from our friends." *(If this is the case, thanks for your honesty but this is plagiarism, results in a warning and no marks. It is preferable to submit no solution at all.)*